

第 12 卷 中关村机试往年题专项与现场拼装卡片

考试速查：主查中关村机试现场策略：规则避坑、三题节奏、往年题信号、部分分路线、满分升级路径和第 01-11 卷翻书清单。

Contents

第 12 卷 中关村机试往年题专项与现场拼装卡片	4
考试速查定位	4
0. 本卷怎么用	4
A. 官方机试规则速查 (D1-D4)	5
D1. 中关村机试规则卡	5
D2. 3 小时节奏卡	5
D3. 32 次提交策略卡	5
D4. 复杂度路由卡	5
B. 往年题总览矩阵	6
B1. 旧卷模块 ID 速查	6
B2. 部分分、升级、满分路线矩阵	6
C. 逐题专项讲解	7
C1. 2025 春 1: 三角形面积	7
题目信号	7
第一反应	7
为什么朴素做法够	7
模块路由	7
部分分方案	7
正解推导	7
关键状态/数据结构	7
易错点	7
组合拼装方案	7
考场写法建议	7
现场迭代路线	8
代码卡片：先交版/正解版	8
C2. 2025 春 2: 路由器优先级调度	8
题目信号	8
第一反应	8
为什么朴素做法不够	8
模块路由	8
部分分方案	8
正解推导	8
关键状态/数据结构	9
易错点	9
组合拼装方案	9
考场写法建议	9
现场迭代路线	9
代码卡片 A: 部分分 $O(nK)$ 线性扫描	9
代码卡片 B: 主力版 priority_queue	10
C3. 2025 春 3: 自动驾驶车队调度	11
题目信号	11
第一反应	11
为什么朴素做法不够	11
模块路由	11
部分分方案	11
正解推导	11
关键状态/数据结构	11

易错点	12
组合拼装方案	12
考场写法建议	12
现场迭代路线	12
代码卡片 A: 部分分 DFS 枚举区域	12
代码卡片 B: 升级版集合 DP / 记忆化搜索	13
代码卡片 C: 弱部分分贪心兜底	14
C4. 2025 夏 1: 线性递推超过阈值	15
题目信号	15
第一反应	15
为什么朴素做法够	15
模块路由	15
部分分方案	15
正解推导	16
关键状态/数据结构	16
易错点	16
组合拼装方案	16
考场写法建议	16
现场迭代路线	16
代码卡片: 先交版/正解版	16
C5. 2025 夏 2: 最优汇率转换	17
题目信号	17
第一反应	17
为什么朴素做法不够	17
模块路由	17
部分分方案	17
正解推导	17
关键状态/数据结构	17
易错点	17
组合拼装方案	17
考场写法建议	17
现场迭代路线	17
代码卡片 A: 部分分直接比较	17
代码卡片 B: 正解 eps 比较	18
C6. 2025 夏 3: 虫洞穿梭	18
题目信号	18
第一反应	18
为什么朴素做法不够	18
模块路由	18
部分分方案	18
正解推导	18
关键状态/数据结构	19
易错点	19
组合拼装方案	19
考场写法建议	19
现场迭代路线	19
代码卡片 A: 部分分 Floyd 小图版	19
代码卡片 B: 部分分 Bellman-Ford 边表版	20
代码卡片 C: 主力版 Dijkstra	21
C7. 2025 秋 1: 字符串嵌入判定	22
题目信号	22
第一反应	22
为什么朴素做法不够	22
模块路由	23
部分分方案	23
正解推导	23
关键状态/数据结构	23
易错点	23
组合拼装方案	23
考场写法建议	23
现场迭代路线	23
代码卡片 A: 部分分只检查长度	23
代码卡片 B: 部分分只检查同一小写一致性	24

代码卡片 C: 正解一致性 + 单射	24
C8. 2025 秋 2: 药液比例最大化	25
题目信号	25
第一反应	25
为什么朴素做法不够	25
模块路由	25
部分分方案	25
正解推导	25
关键状态/数据结构	26
易错点	26
组合拼装方案	26
考场写法建议	26
现场迭代路线	26
代码卡片 A: 部分分 DFS 枚举非增序列	26
代码卡片 B: 部分分小 M 朴素二分 + DP 判定	27
代码卡片 C: 部分分 $b=1$ 特殊 DP	28
代码卡片 D: 正解二分答案 + DP 判定	29
C9. 2025 秋 3: 试剂资源实验选择	30
题目信号	30
第一反应	30
为什么朴素做法不够	30
模块路由	30
部分分方案	31
正解推导	31
关键状态/数据结构	31
易错点	31
组合拼装方案	31
考场写法建议	31
现场迭代路线	31
代码卡片 A: 部分分 DFS 选/不选	31
代码卡片 B: 部分分 $N=1$ 一维背包	32
代码卡片 C: 部分分 $N=2$ 二维背包	33
代码卡片 D: 主力稀疏状态 + 支配删除	33
C10. 2026 冬 1: 图片存储空间计算	35
题目信号	35
第一反应	35
为什么朴素做法有风险	35
模块路由	35
部分分方案	35
正解推导	35
关键状态/数据结构	35
易错点	35
组合拼装方案	35
考场写法建议	35
现场迭代路线	35
代码卡片 A: 部分分 long long 简洁版	35
代码卡片 B: 正解 <code>__int128</code> 防溢出版	36
C11. 2026 冬 2: 环形公路种树与连续段查询	37
题目信号	37
第一反应	37
为什么朴素做法不够	37
模块路由	37
部分分方案	37
正解推导	37
关键状态/数据结构	38
易错点	38
组合拼装方案	38
考场写法建议	38
现场迭代路线	38
代码卡片 A: 部分分数组模拟	38
代码卡片 B: 主力坐标压缩 + 懒标记线段树	39
C12. 2026 冬 3: 虚拟货币兑换的最小手续费	43
题目信号	43

第一反应	43
为什么朴素做法不够	43
模块路由	43
部分分方案	43
正解推导	43
关键状态/数据结构	44
易错点	44
组合拼装方案	44
考场写法建议	44
现场迭代路线	44
代码卡片 A: 部分分 $N \leq 3$ 枚举小环	44
代码卡片 B: 部分分 $p=0$ 最大乘积 Floyd	45
代码卡片 C: 正解二分手续费 + log Floyd	45
D. 专项算法卡片 (D1-D14)	46
D5. 分数规划卡	46
D6. 套利与 log 图卡	47
D7. 环形区间线段树卡	47
D8. 字符映射/单射卡	47
D9. 混合输入解析卡	48
D10. 浮点比较卡	48
D11. 大整数/溢出卡	48
D12. 题面矛盾处理卡	48
D13. 部分分优先卡	49
D14. 最后提交检查卡	49
E. 现场拼装总表	49
F. 最后 30 分钟救分卡片	49
F1. 如果第三题完全没做	49
F2. 如果 WA	50
F3. 如果 TLE	50
F4. 如果 RE	50
F5. 如果最后只剩 5 分钟	50
G. 题内代码索引	50

第 12 卷 中关村机试往年题专项与现场拼装卡片

考试速查定位

第 12 卷 中关村机试往年题专项与现场拼装卡片

- **这是第几卷：** 12。
- **主要查什么：** 主查中关村机试现场策略：规则避坑、三题节奏、往年题信号、部分分路线、满分升级路径和第 01-11 卷翻书清单。
- **什么时候翻：** 考前或开考后需要确认机试规则、往年题信号、部分分路线和翻书拼装清单时翻。
- **题面关键词：** 机试规则；3 小时节奏；往年题；部分分；满分升级；模块拼装；翻书路线。

定位：这是 v0.31 之后追加的考场前插页。它不再泛泛讲算法，而是把官方机试规则、往年题信号、部分分路线、满分升级路线和“翻哪本书”拼在一起。考试时先看题目属于哪一类，再按本卷的模块拼装清单去翻第 01-11 卷。

0. 本卷怎么用

1. 先看“规则卡”和“3 小时节奏卡”，确认不要踩文件 IO、网络、AI、#pragma、输出多余信息这些零分坑。
2. 扫三题后，立刻在“往年题总览矩阵”里找相似题。
3. 找到相似题后，只读四块：模块路由、部分分方案、组合拼装方案、考场写法建议。
4. 先写一个能提交的版本，再翻旧卷升级。不要一开始就追满分。
5. 最后 30 分钟只查卡片，不做大改。

A. 官方机试规则速查 (D1-D4)

D1. 中关村机试规则卡

- 来源：官方机试大纲。本卷是规则摘要，实际情况以考试现场通知为准。
- 线下机考，3 小时，共 3 题。
- 内容：编程基础、算法与数据结构。
- 编程语言：C/C++、Java、Python；主力建议选 g++ with std17。
- 评测环境含 Ubuntu 22.04、g++ 11.4.0 / g++-12 12.3.0、Python 3.10.12。
- 可参考提前准备好的纸质资料。
- 禁止搜索引擎、联网资料、生成式 AI、Copilot 等联网代码补全。
- 禁止文件 IO，程序必须使用标准输入输出。
- 禁止 #pragma 改变编译器参数，禁止内联汇编等系统相关技巧。
- Linux real time 计时，IO 操作也算时间。
- 每题最多 32 次有效提交，编译失败返还次数，WA/RE/TLE 会消耗有效次数。
- 每题最终得分取所有提交中的最高分。
- 输出多余信息、数组越界、空间开爆、死循环、访问非常见系统调用，都可能直接丢分。

现场含义：

- 每题都应该尽早有一次可运行提交。
- 不会正解时，合法输出 + 小数据精确 + 特判，也比空着强。
- 纸质资料要服务“快速拼装”，不要让自己陷在证明和长模板里。

D2. 3 小时节奏卡

时间	目标	现场动作
0-10 分钟	全卷扫描	看三题关键词、数据范围、样例、输出格式，标出“保底题/主攻题/部分分题”。
10-35 分钟	第一题保底	签到题或模拟题先拿分，重点查第 01 卷输入输出、第 11 卷常识。
35-95 分钟	第二题主攻	常见是堆、Dijkstra、DP、线段树。先交朴素，再升级。
95-155 分钟	第三题拿分	先写 DFS/Floyd/数组模拟/朴素 DP，再考虑满分结构。
155-175 分钟	回头补漏	查 long long、下标、闭区间、清空数组、浮点 eps、输出格式。
175-180 分钟	确认最高分版本	不做大改，只确认每题至少有有效提交。

D3. 32 次提交策略卡

- Version 0：输入读对，输出格式合法，特判样例。
- Version 1：小数据暴力，能过一批测试点就交。
- Version 2：加剪枝、记忆化、Floyd、小容量 DP、数组模拟。
- Version 3：换成满分算法。
- 编译失败虽然返还次数，但会浪费现场时间。提交前先本地编译。
- 已得分代码不要在最后 10 分钟彻底推翻。

D4. 复杂度路由卡

数据范围信号	优先算法
$N \leq 20$	DFS、子集枚举、状压 DP。
$N \leq 25$ 且容量很小	枚举小集合、搜索剪枝、集合 DP。
$N \leq 40$	折半枚举、状态压缩谨慎使用。
$N \leq 100$	Floyd、 $O(N^3)$ DP、Bellman-Ford 正环检测。
$N \leq 500$	$O(N^3)$ 要看时限，优先优化。
$N \leq 1000$	$O(N^2)$ 常可接受。
$N, M \leq 1e5/2e5$	$O((N+M) \log N)$ 、线性、堆、线段树。
坐标到 $1e9$ 但操作数有限	坐标压缩、动态开点、map 维护区间。

B. 往年题总览矩阵

题卡	主题	题目信号	首选模块/专项卡
C1	三角形面积	三边、面积、保留 2 位、Invalid	第 11 卷常识; 第 01 卷格式输出
C2	路由器优先级调度	到达时间递增、最高优先级、异常退出	CPP-004; 第 02 卷模拟
C3	自动驾驶车队调度	分配、容量、非空车队数量、大权值	第 02 卷搜索; 第 03 卷 DP
C4	线性递推超过阈值	$x=a*x+b$ 、首次超过	第 01 卷 IO; 第 11 卷边界
C5	最优汇率转换	浮点乘积、输出 0/1/2	D10 浮点比较卡
C6	虫洞穿梭	$u \rightarrow v$ / $u \rightarrow v$ 混合、正权最短路	CPP-011; GRAPH-03
C7	字符串嵌入判定	映射一致、不同小写不同大写	D8 字符映射/单射卡
C8	药液比例最大化	最大化 sumA/sumB、单调前缀	D5 分数规划卡; 第 03 卷 DP
C9	试剂资源实验选择	多资源、选或不选、最大价值	DP-24; 第 02 卷 DFS
C10	图片存储空间计算	bit/byte/KB/MB/GB、四舍五入	D11 大整数/溢出卡; 第 11 卷
C11	环形公路种树	环形区间、区间赋值、最长连续同类型	D7 环形区间线段树卡
C12	虚拟货币手续费	乘积套利、手续费、输出六位	D6 套利与 log 图卡

B1. 旧卷模块 ID 速查

考试时不要只记“第几卷”，还要知道大概翻哪个模块编号：

主题	优先翻阅模块
C++ 骨架/输入输出 string / getline / STL	CPP-001-main-io、CPP-10-io-formatting CPP-011-string-reference、 CPP-013-stl-containers-reference
priority_queue / 堆	CPP-004-queues-stacks-heaps、 DS-07-stl-first-data-structures
暴力、DFS、记忆化	BRUTE-00-partial-score-strategy、 BRUTE-04-combination-dfs、 BRUTE-07-memoized-search-overview
背包/状压/DP 建模	DP-00-total-flow、DP-06-01-knapsack、 DP-16-bitmask-dp、DP-24-knapsack-variants
坐标压缩 线段树/懒标记 Dijkstra Floyd / Bellman-Ford 浮点、几何、单位常识	CPP-007-coordinate-compression DS-03-segtree-sparse、DS-05-advanced-segtree GRAPH-03-dijkstra-path-multisource GRAPH-04-floyd-bellman-spfa MATHREF-06-geometry-numeric-modeling、 SIGN-MEDIA-02-media-format-compression、 SIGN-MATH-01-elementary-formulas
调试/提交检查	TRAIN-00-debug-checklist

B2. 部分分、升级、满分路线矩阵

题目	先交部分分	升级路线	满分路线
三角形面积 路由器调度	合法性 + 海伦公式 $O(nK)$ 线性找最高优先级	固定两位输出 指针扫描到达事件	$O(1)$ 正解 priority_queue + sent[id] + 原顺序输出
自动驾驶车队	区域 DFS、小 n/m 状压	预处理每车队可选小集合、剪枝上界	若奖励可线性化用费用流; 否则用集合 DP/分支限界冲高分, 现场以数据特征决定
线性递推 汇率转换 虫洞穿梭 字符串嵌入 药液比例	while 模拟 直接乘积比较 小图 Floyd/Bellman-Ford 长度/一致性检查 DFS 枚举 k_i	long long 防溢出 eps 判断相等 混合输入解析 加 used[26] $a-mid*b$ 判定	$O(\text{次数})$ 正解 $O(1)$ 正解 邻接表 + 堆优化 Dijkstra mp[26] + used[26] 单射正解 二分答案 + 单调前缀 DP

题目	先交部分分	升级路线	满分路线
试剂选择	DFS 选/不选	容量小用多维背包	依据规模选多维 DP、稀疏状态支配删除或折半
图片大小	long long 计算	__int128 中间乘法	单位阈值 + 指定舍入
环形种树	数组模拟	map 区间维护	坐标压缩 + lazy segment tree + 环形顺序 merge
虚拟货币	枚举小环 / p=0 检测	log 图检测正环	二分手续费 + Floyd/Bellman-Ford 正环判定

C. 逐题专项讲解

C1. 2025 春 1: 三角形面积

题目信号

- 输入三个整数 a, b, c 。
- 输出面积，保留两位。
- 不合法输出 Invalid。
- 数据范围小，是典型签到题。

第一反应

直接判断三角形合法性，再用海伦公式：

```
s = (a+b+c)/2
area = sqrt(s*(s-a)*(s-b)*(s-c))
```

为什么朴素做法够

只有三个数， $O(1)$ 。唯一风险是合法性和输出格式。

模块路由

- 第 01 卷：double、fixed << setprecision(2)。
- 第 06 卷：几何和 sqrt。
- 第 11 卷：三角形面积、单位和常识题。

部分分方案

- 只判断样例和正常三角形，也能过部分。
- 加上三角形不等式后基本满分。

正解推导

能构成三角形的条件是任意两边之和大于第三边。因为边长为正整数，只要排序后检查 $x+y>z$ 即可。合法后使用海伦公式。

关键状态/数据结构

无状态，使用 double。

易错点

- 非法三角形不能继续 sqrt，否则可能出现负数开根。
- 保留两位不是“有效数字”，实现上按样例应使用固定小数位。
- Invalid 大小写要完全一致。

组合拼装方案

复制第 01 卷格式输出骨架，加入第 11 卷海伦公式卡片。

考场写法建议

5 分钟内写完，样例、1 1 2、3 4 5、2 2 3 各测一次。不要在第一题浪费时间。

现场迭代路线

这题的真实策略是“不要想复杂”。看到三边和面积，第一反应就是海伦公式；真正会丢分的不是算法，而是非法三角形和格式输出。考场上先写合法性判断，再写公式，最后补 `fixed << setprecision(2)`。如果样例是非法三角形，先保证输出字符串完全一致。

代码卡片：先交版/正解版

本题没有有意义的复杂升级，先交版就是正解。重点是非法三角形和固定两位输出。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a[4];
    if (!(cin >> a[1] >> a[2] >> a[3])) return 0;
    sort(a + 1, a + 4);

    if (a[1] <= 0 || (__int128)a[1] + a[2] <= a[3]) {
        cout << "Invalid\n";
        return 0;
    }

    double x = a[1], y = a[2], z = a[3];
    double s = (x + y + z) / 2.0;
    double area = sqrt(max(0.0, s * (s - x) * (s - y) * (s - z)));
    cout << fixed << setprecision(2) << area << "\n";
    return 0;
}
```

C2. 2025 春 2：路由器优先级调度

题目信号

- n 个包，到达时间递增。
- 每次从“已到达但未发送”的包里取优先级最高。
- 发送 K 个后异常退出。
- 剩余包按原到达顺序输出。

第一反应

每次发送前线性扫描所有包，找到到达时间不超过当前时间且未发送的最高优先级包。

为什么朴素做法不够

$n \leq 1000$ 时 $O(nK)$ 也许能过；如果同类题放到 $1e5$ ，线性扫描会爆。纸质资料应该同时准备线性扫描和堆版本。

模块路由

- 第 01 卷：priority_queue。
- 第 02 卷：模拟、部分分。
- 第 04 卷：堆。
- 第 07 卷：输出剩余状态检查。

部分分方案

1. $O(nK)$ 线性扫描：小 n 可过。
2. 忽略异常后顺序，只模拟发送顺序：可能拿不到输出剩余要求的点，但能帮调试。

正解推导

到达时间递增，所以用指针 `ptr` 从左到右扫描。每次发送前，把所有已到达包加入堆；如果堆为空且还有未来包，就把当前时间跳到下一个到达时间，再继续加入。堆按优先级从高到低弹出。弹出后标记 `sent[id]=true`。异常退出后，默认按“所有未发送数据包”理解，从 1 到 n 扫描输出 `sent[id]==false` 的优先级；如果现场题面明确写“路由器缓存中剩余包”，才额外过滤“已经到达”的包。

关键状态/数据结构

- `priority_queue<pair<int,int>>` `pq`: 第一关键字优先级, 第二关键字可放负编号或编号。
- `sent[i]`: 是否已发送。
- `priority[i]`、`arrive[i]`: 原始数组。
- `ptr`: 下一个未加入堆的包。

易错点

- 剩余包输出按到达顺序, 不是堆顺序。
- 若 $K=n$, 输出 Normal。
- 堆里不要只存优先级, 否则最后无法标记原始编号。
- “剩余包”有两种可能: 所有未发送包, 或已经到达但未发送包。原整理题面偏向“未发送包按到达顺序”, 若现场原文写缓存/队列, 再按已到达过滤。

组合拼装方案

- CPP-004-queues-stacks-heaps: 复制 `priority_queue<pair<int,int>>` 最大堆用法。
- BRUTE-13-small-exact-large-special: 先写 $O(nK)$ 小数据扫描。
- TRAIN-00-debug-checklist: 用边界样例检查 $K=0$ 、 $K=n$ 、堆为空跳到下次到达、优先级升序/降序。

考场写法建议

先写线性扫描, 15 分钟可交; 若第二题数据较大, 再换堆。堆版本完成后, 用同一批小随机数据和线性版本对拍。

现场迭代路线

先把题目想成“时间轴模拟”: 当前时间 `now`, 哪些包已经到达, 选其中优先级最高的发送。第一版用 $O(nK)$ 扫描, 因为它最贴近题意, 便于确认剩余包输出规则。确认规则后再升级堆: 到达事件只入堆一次, 发送时弹堆顶。最后专门处理异常退出后的剩余输出, 因为这一步和调度算法本身是两件事。

代码卡片 A: 部分分 $O(nK)$ 线性扫描

适用: 小数据、先交版本。每次发送都重新扫描所有包, 最容易确认题意。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, K;
    if (!(cin >> n >> K)) return 0;

    vector<long long> pri(n + 1), arrive(n + 1);
    for (int i = 1; i <= n; i++) cin >> pri[i];
    for (int i = 1; i <= n; i++) cin >> arrive[i];

    vector<int> sent(n + 1, 0);
    long long now = (n >= 1 ? arrive[1] : 0);
    int done = 0;

    while (done < K && done < n) {
        int best = 0;
        for (int i = 1; i <= n; i++) {
            if (sent[i] || arrive[i] > now) continue;
            if (best == 0 || pri[i] > pri[best] || (pri[i] == pri[best] && i < best)) {
                best = i;
            }
        }

        if (best == 0) {
            long long nxt = (1LL << 60);
            for (int i = 1; i <= n; i++) {
                if (!sent[i] && arrive[i] > now) nxt = min(nxt, arrive[i]);
            }
        }
    }
```

```

    }
    if (nxt == (1LL << 60)) break;
    now = nxt;
    continue;
}

sent[best] = 1;
done++;
now++;
}

bool any = false;
for (int i = 1; i <= n; i++) {
    if (!sent[i]) {
        if (any) cout << ' ';
        cout << pri[i];
        any = true;
    }
}
if (!any) cout << "Normal";
cout << "\n";
return 0;
}

```

代码卡片 B: 主力版 priority_queue

适用: n 或 K 大时。堆中存 {优先级, -原编号}, 弹出后用原编号标记已发送。

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, K;
    if (!(cin >> n >> K)) return 0;

    vector<long long> pri(n + 1), arrive(n + 1);
    for (int i = 1; i <= n; i++) cin >> pri[i];
    for (int i = 1; i <= n; i++) cin >> arrive[i];

    priority_queue<pair<long long, int>> pq;
    vector<int> sent(n + 1, 0);

    long long now = (n >= 1 ? arrive[1] : 0);
    int ptr = 1;
    int done = 0;

    while (done < K && done < n) {
        while (ptr <= n && arrive[ptr] <= now) {
            pq.push({pri[ptr], -ptr});
            ptr++;
        }

        if (pq.empty()) {
            if (ptr > n) break;
            now = max(now, arrive[ptr]);
            continue;
        }

        int id = -pq.top().second;
        pq.pop();
        sent[id] = 1;
    }
}

```

```

        done++;
        now++;
    }

    bool any = false;
    for (int i = 1; i <= n; i++) {
        if (!sent[i]) {
            if (any) cout << ' ';
            cout << pri[i];
            any = true;
        }
    }
    if (!any) cout << "Normal";
    cout << "\n";
    return 0;
}

```

C3. 2025 春 3: 自动驾驶车队调度

题目信号

- 区域分配给车队。
- 每个区域最多由一个车队负责。
- 每个车队最多负责 k 个区域, 且 $k \leq 5$ 。
- 被服务区域数量在 $[L, R]$ 。
- $A[i][cnt]$ 和 $B[region][team]$ 可到 $1e9$, 有负数。
- 题面提到 t , 但输入没有 t 。

第一反应

DFS 按区域枚举: 每个区域选择不分配或分给某个车队, 维护每个车队已分配数量和总收益。

为什么朴素做法不够

区域数和车队数最多 25, 直接 $(m+1)^n$ 爆炸。前 30% $n, m \leq 8$ 可以用 DFS, 前 60% 可以尝试剪枝或状压, 满分需要把容量小、非空数量和区域互斥读进状态。

模块路由

- 第 02 卷: DFS、剪枝、记忆化、部分分。
- 第 03 卷: 集合 DP、容量约束 DP、从约束读状态。
- 第 04 卷: 如果走费用流需要图建模辅助。
- 第 05 卷: 费用流/匹配方向可参考图论低优先级模块。
- 第 07 卷: 大权值和负无穷检查。

部分分方案

1. $n, m \leq 8$: DFS 枚举每个区域分给哪个车队或不分配。
2. $n \leq 20$: 区域集合状压, 枚举每个车队负责的区域子集, 子集大小不超过 k 。
3. 贪心兜底: 按 $B[j][i]$ 最大分配, 但只能作为弱部分分, 不能保证最优。

正解推导

关键是 $k \leq 5$, 每个车队能拿的区域子集很小。对每个车队 i , 预先枚举所有大小 $0 \dots k$ 的区域集合 S , 贡献为:

$$\text{gain}(i, S) = A[i][|S|] + \sum B[\text{region}][i]$$

然后在车队维度做 DP: 前 i 个车队已经覆盖了哪些区域、用了多少非空车队或最终服务数量是否落在 $[L, R]$ 。如果 $n \leq 25$, 完整 2^n 很大, 满分可能需要更强剪枝、分支限界、费用流或根据原题隐藏数据特征选择方案。开卷资料的目标是让你至少能写出前 30%-60% 的可靠版本, 并有升级方向。

关键状态/数据结构

- DFS 状态: $\text{posRegion}, \text{cntTeam}[1..m], \text{usedTeams}, \text{currentScore}$ 。
- 状压 DP 状态: $\text{dp}[i][\text{mask}]$ 表示处理前 i 个车队、已覆盖区域集合为 mask 的最大收益。
- 若需要非空车队数量, 升维为 $\text{dp}[i][\text{mask}][\text{used}]$ 或按 $\text{popcount}(\text{mask})$ 最后筛 $[L, R]$ 。
- 负无穷用 $\text{const long long NEG} = -(1LL < 60)$, 不要用 $-1e18$ 再乱加。

易错点

- 不要读取不存在的 t 。题面矛盾时，以正式输入格式为准。
- $A[i][0]$ 可能不是 0，车队不分配区域也要计入。
- A 、 B 可为负数，不能贪心地“负数就不选”。
- `long long` 必须全程使用。
- L, R 约束到底是“实时服务区域数”还是“分配后非空车队数”，要按现场题面判断；若题面像原稿，应保守把它理解为最终可同时服务数量，即非空车队数与区域数受车队数量共同限制。

组合拼装方案

- 小数据：第 02 卷 DFS + 第 07 卷剪枝调试。
- 中数据：第 03 卷状压 DP + 第 02 卷子集枚举。
- 满分冲刺：先写可得 DP，再考虑费用流或搜索剪枝，不要从空白直接写复杂模型。

考场写法建议

这类题不要第一时间赌满分。先写 $n, m \leq 8$ DFS 并提交；再把 DFS 参数抽象成“车队容量、区域是否被占用”，看能否加记忆化或换成车队选子集 DP。

现场迭代路线

这题的核心不是套某个模板，而是把约束翻译成状态。第一反应按区域 DFS：每个区域“不分配/分给某车队”，这是最自然也最容易先交的版本。然后观察重复信息：未来只关心“哪些区域已被占用”和“处理到第几个车队”，于是升级成车队选子集的集合 DP。若现场题面仍缺参数或规模不完整，先采用本卷合理化模型拿高分，不要因为追一个不确定的满分模型导致整题空着。

代码卡片 A：部分分 DFS 枚举区域

适用： $n, m \leq 8$ 或样例/小数据。每个区域选择“不分配”或分给某个车队，最后检查服务区域数量。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG = -(1LL << 60);

int n, m, k, L, R;
vector<vector<ll>> A, B;
vector<int> cntTeam;
ll bestAns = NEG;

void dfs(int region, int assigned, ll sumB) {
    if (assigned > R) return;
    if (assigned + (n - region + 1) < L) return;

    if (region == n + 1) {
        if (assigned < L || assigned > R) return;
        ll total = sumB;
        for (int team = 1; team <= m; team++) {
            total += A[team][cntTeam[team]];
        }
        bestAns = max(bestAns, total);
        return;
    }

    dfs(region + 1, assigned, sumB);

    for (int team = 1; team <= m; team++) {
        if (cntTeam[team] == k) continue;
        cntTeam[team]++;
        dfs(region + 1, assigned + 1, sumB + B[region][team]);
        cntTeam[team]--;
    }
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m >> k >> L >> R)) return 0;

    A.assign(m + 1, vector<ll>(k + 1, 0));
    for (int team = 1; team <= m; team++) {
        for (int c = 0; c <= k; c++) cin >> A[team][c];
    }

    B.assign(n + 1, vector<ll>(m + 1, 0));
    for (int region = 1; region <= n; region++) {
        for (int team = 1; team <= m; team++) cin >> B[region][team];
    }

    cntTeam.assign(m + 1, 0);
    dfs(1, 0, 0);
    cout << bestAns << "\n";
    return 0;
}

```

代码卡片 B: 升级版集合 DP / 记忆化搜索

合理化版本: 每个区域最多分配给一个车队; 每个车队最多服务 k 个区域; 总服务区域数在 $[L, R]$ 。适用 $n \leq 20 \sim 22$, 或 R 较小、剪枝有效的数据。

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG = -(1LL << 60);

int n, m, k, L, R;
vector<vector<ll>> A, B;
uint32_t fullMask;
vector<unordered_map<uint32_t, ll>> memo;

void genOptions(int team, int start, int chosen, int limit, uint32_t avail,
               uint32_t sub, ll sumB, vector<pair<uint32_t, ll>>& out) {
    out.push_back({sub, A[team][chosen] + sumB});
    if (chosen == limit) return;

    for (int region = start; region <= n; region++) {
        uint32_t bit = 1u << (region - 1);
        if ((avail & bit) == 0) continue;
        genOptions(team, region + 1, chosen + 1, limit, avail,
                  sub | bit, sumB + B[region][team], out);
    }
}

ll dfs(int team, uint32_t usedMask) {
    int served = __builtin_popcount(usedMask);
    if (served > R) return NEG;
    if (served + (m - team + 1) * k < L) return NEG;

    if (team == m + 1) {
        return (L <= served && served <= R) ? 0 : NEG;
    }

    auto& mp = memo[team];
    if (mp.count(usedMask)) return mp[usedMask];

    uint32_t avail = fullMask ^ usedMask;

```

```

vector<pair<uint32_t, ll>> options;
int maxPick = min(k, R - served);
genOptions(team, 1, 0, maxPick, avail, 0, 0, options);

ll ans = NEG;
for (auto [sub, gain] : options) {
    ll nxt = dfs(team + 1, usedMask | sub);
    if (nxt != NEG) ans = max(ans, gain + nxt);
}
return mp[usedMask] = ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m >> k >> L >> R)) return 0;
    if (n > 25) {
        cout << 0 << "\n";
        return 0;
    }

    A.assign(m + 1, vector<ll>(k + 1, 0));
    for (int team = 1; team <= m; team++) {
        for (int c = 0; c <= k; c++) cin >> A[team][c];
    }

    B.assign(n + 1, vector<ll>(m + 1, 0));
    for (int region = 1; region <= n; region++) {
        for (int team = 1; team <= m; team++) cin >> B[region][team];
    }

    fullMask = (1u << n) - 1;
    memo.assign(m + 2, {});
    ll ans = dfs(1, 0);
    cout << ans << "\n";
    return 0;
}

```

代码卡片 C: 弱部分贪心兜底

适用: 完全来不及写 DP 时的合法兜底。每次挑当前还能服务的 B[region][team] 最大组合, 不保证最优。

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Choice {
    ll gain;
    int region, team;
    bool operator<(const Choice& other) const {
        return gain < other.gain;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, k, L, R;
    if (!(cin >> n >> m >> k >> L >> R)) return 0;

    vector<vector<ll>> A(m + 1, vector<ll>(k + 1, 0));

```

```

for (int team = 1; team <= m; team++) {
    for (int c = 0; c <= k; c++) cin >> A[team][c];
}

vector<vector<ll>> B(n + 1, vector<ll>(m + 1, 0));
for (int region = 1; region <= n; region++) {
    for (int team = 1; team <= m; team++) cin >> B[region][team];
}

priority_queue<Choice> pq;
for (int region = 1; region <= n; region++) {
    for (int team = 1; team <= m; team++) pq.push({B[region][team], region, team});
}

vector<int> usedRegion(n + 1, 0), cntTeam(m + 1, 0);
ll sumB = 0;
int assigned = 0;

while (!pq.empty() && assigned < R) {
    auto cur = pq.top();
    pq.pop();
    if (usedRegion[cur.region]) continue;
    if (cntTeam[cur.team] == k) continue;
    usedRegion[cur.region] = 1;
    cntTeam[cur.team]++;
    sumB += cur.gain;
    assigned++;
}

if (assigned < L) {
    cout << 0 << "\n";
    return 0;
}

ll ans = sumB;
for (int team = 1; team <= m; team++) ans += A[team][cntTeam[team]];
cout << ans << "\n";
return 0;
}

```

C4. 2025 夏 1: 线性递推超过阈值

题目信号

- 初值 $x=1$ 。
- 每次 $x = a*x + b$ 。
- 求首次 $x > c$ 的操作次数。

第一反应

while 循环模拟并计数。

为什么朴素做法够

$c \leq 1e6$, $a \geq 1$, $b \geq 0$ 。增长不慢，循环次数很小。如果 $a=1, b=0, c \geq 1$ 则永远不超过，但原题可能没有这种情况，需要现场检查。

模块路由

- 第 01 卷：整数输入输出。
- 第 11 卷：递推和边界常识。

部分分方案

直接模拟就是正解。

正解推导

从 $x=1$ 开始，先判断是否已经 $x>c$ 。若没有，每操作一次更新并计数，直到超过。

关键状态/数据结构

long long x 和 int ans 。

易错点

- 是“超过”不是“大于等于”。
- 先操作再判断还是先判断，要看初值是否可能已经超过。
- $a*x+b$ 用 long long。

组合拼装方案

第 01 卷主骨架即可。

考场写法建议

这是签到题，3-5 分钟内完成。自测 1 1 1、2 3 10 (应输出 2)、10 0 1。

现场迭代路线

先确认答案含义是“操作次数”而不是“第几项”。初值 $x=1$ 已经是第 0 次，只有执行一次递推后才让计数加 1。第一版直接 while 模拟即可；如果看到 $a*x+b$ 可能变大很快，就把中间值换成 __int128，防止超过阈值前后溢出。

代码卡片：先交版/正解版

直接循环就是正解。这里用 __int128 防止 $a*x+b$ 在超过阈值前后溢出。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b, c;
    if (!(cin >> a >> b >> c)) return 0;

    __int128 x = 1;
    long long ans = 0;

    if (x > c) {
        cout << 0 << "\n";
        return 0;
    }

    while (x <= c) {
        __int128 nx = (__int128)a * x + b;
        ans++;
        if (nx > c) break;
        if (nx == x) {
            cout << -1 << "\n";
            return 0;
        }
        x = nx;
    }

    cout << ans << "\n";
    return 0;
}
```

C5. 2025 夏 2: 最优汇率转换

题目信号

- 四个正实数。
- 比较两条两步兑换路径。
- 输出 $1/2/0$ 。
- 题面说相差 $1e-5$ 内视为相等。

第一反应

比较 $r1*r2$ 和 $r3*r4$ 。

为什么朴素做法不够

直接 `==` 比较 `double` 会错。必须用 `eps`。

模块路由

- 第 01 卷: `double` 输入输出。
- 第 07 卷: 浮点误差调试。
- 第 12 卷: 浮点比较卡。

部分分方案

直接乘积比较可过非边界数据; 加 `eps` 后满分。

正解推导

初始 `A` 的数量相同, 最终数量分别乘以 $r1*r2$ 与 $r3*r4$ 。比较两个乘积即可。若差的绝对值小于等于 `eps`, 输出 `0`。

关键状态/数据结构

```
double x = r1 * r2;
double y = r3 * r4;
```

易错点

- 若题面明确“相差 $1e-5$ 内视为相等”, 就用 `const double EPS = 1e-5;`。只有题面没有给相等阈值时, 才按输出精度另取更小 `eps`。
- 不要输出多余小数。

组合拼装方案

- `C++-001-main-io`: 读取四个 `double`。
- `C++-10-io-formatting`: 确认本题不需要小数输出。
- 本卷 `D10` 浮点比较卡: 使用题面 `eps`。

考场写法建议

写完立刻测: 完全相等、差很小、第一组大、第二组大。

现场迭代路线

先把兑换路径化简成两个乘积, 别引入图论或 `DP`。第一版直接比较乘积, 可以马上覆盖普通数据; 随后立刻加 `eps`, 因为题面已经明确“接近相等”的判定规则。考场上这题最重要的是把输出压成整数 $1/2/0$, 不要因为浮点题就输出小数。

代码卡片 A: 部分分直接比较

适用: 非边界数据。这个版本故意不加 `eps`, 考场上可先交但应尽快升级。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    double r1, r2, r3, r4;
```

```

    if (!(cin >> r1 >> r2 >> r3 >> r4)) return 0;

    double x = r1 * r2;
    double y = r3 * r4;
    if (x > y) cout << 1 << "\n";
    else if (x < y) cout << 2 << "\n";
    else cout << 0 << "\n";
    return 0;
}

```

代码卡片 B: 正解 eps 比较

适用: 题面给“相差 $1e-5$ 内视为相等”的完整版本。

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    double r1, r2, r3, r4;
    if (!(cin >> r1 >> r2 >> r3 >> r4)) return 0;

    double x = r1 * r2;
    double y = r3 * r4;
    const double EPS = 1e-5;

    if (fabs(x - y) <= EPS) cout << 0 << "\n";
    else if (x > y) cout << 1 << "\n";
    else cout << 2 << "\n";
    return 0;
}

```

C6. 2025 夏 3: 虫洞穿梭

题目信号

- $N \leq 1e5$, $M \leq 2e5$ 。
- 边权正数。
- 同时有 $u \rightarrow v$ 和 $u \leftarrow v$ 。
- 求 S 到 T 最短路。

第一反应

小图可以 Floyd; 大图必须邻接表 + 堆优化 Dijkstra。

为什么朴素做法不够

Floyd 是 $O(N^3)$, $1e5$ 完全不可用。Bellman-Ford $O(NM)$ 也不可。正权边直接 Dijkstra。

模块路由

- 第 01 卷: getline、stringstream、字符串查找。
- 第 05 卷: Dijkstra、统一 1-index 建图。
- 第 07 卷: 不可达、long long、混合输入调试。

部分分方案

1. 所有边都是双向且 $N \leq 100$: Floyd。
2. 小图混合有向/无向: Bellman-Ford。
3. 满分: 解析后建邻接表, Dijkstra。

正解推导

边权均为正, 因此单源最短路可用 Dijkstra。难点不是算法, 而是输入解析: 如果一行包含 $\rightarrow$, 就是单向边; 否则是三个整数, 表示双向边。

关键状态/数据结构

- `vector<pair<int,int>> g[MAXN]` 或静态边表。
- `dist[i]` 使用 `long long`。
- `priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>>`。

解析片段：需要包含 `<string>`、`<sstream>`，并已经定义 `add_edge(u,v,t)`。

```
string line;
getline(cin, line);
if (line.find("->") != string::npos) {
    size_t pos = line.find("->");
    int u = stoi(line.substr(0, pos));
    string rest = line.substr(pos + 2);
    stringstream ss(rest);
    int v, t;
    ss >> v >> t;
    add_edge(u, v, t);
} else {
    stringstream ss(line);
    int u, v, t;
    ss >> u >> v >> t;
    add_edge(u, v, t);
    add_edge(v, u, t);
}
```

易错点

- `cin >> N >> M >> S >> T` 后要清掉换行再 `getline`。
- `u->v t` 里 `u` 和 `v` 可能没有空格，不能只用 `cin >>`。
- 不可达输出 `-1`。
- Dijkstra 只适合非负边；本题 `t>=1`。

组合拼装方案

- 第 01 卷复制 `getline + stringstream`。
- 第 05 卷复制 Dijkstra。
- 改接口：统一 `add_edge(u,v,w)`，解析层只负责决定加一条还是两条边。

考场写法建议

先写解析，把每条边打印到草稿上人工核对，不要在正式提交里输出调试。Dijkstra 使用旧卷模板，不要现场重写堆细节。

现场迭代路线

这题先解决输入，再解决算法。第一步只写解析，把 `u v t` 和 `u->v t` 都转成统一的边表；小图用 Floyd 跑通，能最快暴露解析错误。第二步如果图稍大，Bellman-Ford 仍然用同一份边表，适合先交部分分。最后确认边权全为正，再把边表改邻接表接 Dijkstra，这样升级时不用重想题意。

代码卡片 A：部分分 Floyd 小图版

适用： $N \leq 100$ 小图。优点是代码短，不需要堆；缺点是 $O(N^3)$ 。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = (1LL << 60);

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, S, T;
    if (!(cin >> n >> m >> S >> T)) return 0;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```

vector<vector<ll>> d(n + 1, vector<ll>(n + 1, INF));
for (int i = 1; i <= n; i++) d[i][i] = 0;

for (int i = 1; i <= m; ) {
    string line;
    getline(cin, line);
    if (line.find_first_not_of(" \\t\\r\\n") == string::npos) continue;

    size_t pos = line.find("->");
    if (pos != string::npos) {
        int u = stoi(line.substr(0, pos));
        stringstream ss(line.substr(pos + 2));
        int v;
        ll w;
        ss >> v >> w;
        d[u][v] = min(d[u][v], w);
    } else {
        stringstream ss(line);
        int u, v;
        ll w;
        ss >> u >> v >> w;
        d[u][v] = min(d[u][v], w);
        d[v][u] = min(d[v][u], w);
    }
    i++;
}

for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        if (d[i][k] == INF) continue;
        for (int j = 1; j <= n; j++) {
            if (d[k][j] == INF) continue;
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}

cout << (d[S][T] == INF ? -1 : d[S][T]) << "\\n";
return 0;
}

```

代码卡片 B: 部分 Bellman-Ford 边表版

适用: 小中图, 混合有向/无向都能处理; 正权图也可用, 只是慢。

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = (1LL << 60);

struct Edge {
    int u, v;
    ll w;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, S, T;
    if (!(cin >> n >> m >> S >> T)) return 0;
    cin.ignore(numeric_limits<streamsize>::max(), '\\n');

```

```

vector<Edge> edges;
for (int i = 1; i <= m; ) {
    string line;
    getline(cin, line);
    if (line.find_first_not_of(" \t\r\n") == string::npos) continue;

    size_t pos = line.find("->");
    if (pos != string::npos) {
        int u = stoi(line.substr(0, pos));
        stringstream ss(line.substr(pos + 2));
        int v;
        ll w;
        ss >> v >> w;
        edges.push_back({u, v, w});
    } else {
        stringstream ss(line);
        int u, v;
        ll w;
        ss >> u >> v >> w;
        edges.push_back({u, v, w});
        edges.push_back({v, u, w});
    }
    i++;
}

vector<ll> dist(n + 1, INF);
dist[S] = 0;
for (int round = 1; round <= n - 1; round++) {
    bool changed = false;
    for (const auto& e : edges) {
        if (dist[e.u] == INF) continue;
        if (dist[e.v] > dist[e.u] + e.w) {
            dist[e.v] = dist[e.u] + e.w;
            changed = true;
        }
    }
    if (!changed) break;
}

cout << (dist[T] == INF ? -1 : dist[T]) << "\n";
return 0;
}

```

代码卡片 C: 主力版 Dijkstra

适用: 正权大图。解析层决定加一条边还是两条边, 算法层只处理邻接表。

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = (1LL << 62);

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, S, T;
    if (!(cin >> n >> m >> S >> T)) return 0;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    vector<vector<pair<int, ll>>> g(n + 1);

    for (int i = 1; i <= m; ) {

```

```

string line;
getline(cin, line);
if (line.find_first_not_of(" \t\r\n") == string::npos) continue;

size_t pos = line.find("->");
if (pos != string::npos) {
    int u = stoi(line.substr(0, pos));
    stringstream ss(line.substr(pos + 2));
    int v;
    ll w;
    ss >> v >> w;
    g[u].push_back({v, w});
} else {
    stringstream ss(line);
    int u, v;
    ll w;
    ss >> u >> v >> w;
    g[u].push_back({v, w});
    g[v].push_back({u, w});
}
i++;
}

vector<ll> dist(n + 1, INF);
priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;

dist[S] = 0;
pq.push({0, S});

while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();
    if (d != dist[u]) continue;

    for (auto [v, w] : g[u]) {
        if (dist[v] > d + w) {
            dist[v] = d + w;
            pq.push({dist[v], v});
        }
    }
}

cout << (dist[T] == INF ? -1 : dist[T]) << "\n";
return 0;
}

```

C7. 2025 秋 1: 字符串嵌入判定

题目信号

- 小写字符串 s 。
- 多个大写候选 T 。
- 要存在映射 $a-z \rightarrow A-Z$ 。
- 同一个小写必须映射同一个大写，不同小写不能映射同一个大写。

第一反应

逐字符检查长度和对应字符。

为什么朴素做法不够

只检查同一个小写的一致性还不够，还要检查不同小写不能映射到同一大写，也就是单射约束。

模块路由

- 第 01 卷: string。
- 第 06 卷: 字符串基础。
- 第 12 卷: 字符映射/单射卡。

部分分方案

1. 只检查长度, 拿极弱分。
2. 只检查 mp[26] 一致性, 可过没有冲突单射的点。
3. 加 used[26] 后满分。

正解推导

对每个候选串重新清空:

- mpFrom[26] = -1: 某小写已经映射到哪个大写。
- usedTo[26] = false: 某大写是否已被其他小写占用。

扫描每个位置:

- 如果 mpFrom[x] == -1, 则要求 usedTo[y] == false, 然后建立映射。
- 如果 mpFrom[x] != -1, 则必须等于 y。

关键状态/数据结构

```
int mp[26];
bool used[26];
```

易错点

- 每个候选串都要重新初始化。
- 样例截断时不要依赖样例行数推额外规则。
- 如果候选串长度不等于 N, 直接不合法。

组合拼装方案

第 01 卷 string 输入 + 本卷单射卡。

考场写法建议

这题像签到题, 但很容易漏单射。写完测 ab -> XX 应为不合法, aa -> XY 应为不合法, aba -> XYX 应为合法。

现场迭代路线

先用反例驱动建模: aa -> XY 说明同一小写必须映射一致, ab -> XX 说明不同小写不能抢同一个大写。第一版只查长度, 第二版加 mp[26] 检查一致性, 最终再加 used[26] 检查单射。每个候选串独立判断, 所以所有映射数组都必须在候选内部重置。

代码卡片 A: 部分分只检查长度

适用: 极弱保底。合理化版本输出合法候选数量。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M;
    string s;
    if (!(cin >> N >> M)) return 0;
    cin >> s;

    int ans = 0;
    for (int i = 1; i <= M; i++) {
        string t;
        cin >> t;
```

```

        if ((int)t.size() == N) ans++;
    }
    cout << ans << "\n";
    return 0;
}

```

代码卡片 B: 部分分只检查同一小写一致性

适用: 没有单射冲突的数据。会误判 ab -> XX。

```

#include <bits/stdc++.h>
using namespace std;

bool ok(const string& s, const string& t) {
    if (s.size() != t.size()) return false;
    int mp[26];
    fill(mp, mp + 26, -1);

    for (int i = 0; i < (int)s.size(); i++) {
        int x = s[i] - 'a';
        int y = t[i] - 'A';
        if (mp[x] == -1) mp[x] = y;
        else if (mp[x] != y) return false;
    }
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M;
    string s;
    if (!(cin >> N >> M)) return 0;
    cin >> s;

    int ans = 0;
    for (int i = 1; i <= M; i++) {
        string t;
        cin >> t;
        if (ok(s, t)) ans++;
    }
    cout << ans << "\n";
    return 0;
}

```

代码卡片 C: 正解一致性 + 单射

适用: 完整版本。每个候选串必须重新初始化 mp 和 used。

```

#include <bits/stdc++.h>
using namespace std;

bool ok(const string& s, const string& t) {
    if (s.size() != t.size()) return false;

    int mp[26];
    bool used[26];
    fill(mp, mp + 26, -1);
    fill(used, used + 26, false);

    for (int i = 0; i < (int)s.size(); i++) {
        int x = s[i] - 'a';
        int y = t[i] - 'A';

```

```

        if (mp[x] == -1) {
            if (used[y]) return false;
            mp[x] = y;
            used[y] = true;
        } else {
            if (mp[x] != y) return false;
        }
    }
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M;
    string s;
    if (!(cin >> N >> M)) return 0;
    cin >> s;

    int ans = 0;
    for (int i = 1; i <= M; i++) {
        string t;
        cin >> t;
        if (ok(s, t)) ans++;
    }
    cout << ans << "\n";
    return 0;
}

```

C8. 2025 秋 2: 药液比例最大化

题目信号

- 最大化 $\text{sumA} / \text{sumB}$ 。
- 每个容器只能取底部前缀若干层。
- 总层数必须等于 K 。
- $k_i \geq k_{i+1}$ 单调约束。
- 输出浮点，误差 $1e-4$ 。

第一反应

枚举每个容器取几层，计算比值最大。

为什么朴素做法不够

$N \leq 40$, $M \leq 500$, $N * M \leq 1000$, 枚举所有 k_i 是指数级。需要把“最大比值”转成“判定某个比值能否达到”。

模块路由

- 第 02 卷：小规模 DFS 和枚举。
- 第 03 卷：DP 建模、背包/前缀选择。
- 第 12 卷：分数规划卡。
- 第 07 卷：浮点二分和样例矛盾处理。

部分分方案

1. $N \leq 3$: DFS 枚举所有非增序列 $k_1 \geq \dots \geq k_N$, 且总和 K 。
2. M 小: 二分答案 + 朴素 DP 判定, 状态为处理到容器 i 、已取层数 sum 、上一容器取了 prev 。
3. $b_i, j=1$: 目标变成最大总 A , 可直接 DP。

正解推导

最大化:

```
sumA / sumB >= mid
等价于 sumA - mid * sumB >= 0
```

所以二分答案 mid 。对每层定义贡献 $val = a - mid * b$ 。问题变成：在约束下选 K 层，使总 val 最大，判断最大值是否非负。

因为每个容器只能取底部前缀，预处理每个容器取 j 层的前缀贡献 $pref[i][j]$ 。又有 $k_i > k_{i+1}$ ，DP 时要保留上一容器取了多少层，或者按层数单调从大到小转移。

一种初学者可写的判定：

$dp[i][sum][last]$ = 前 i 个容器，总取 sum 层，第 i 个容器取 $last$ 层时的最大贡献
转移到第 $i+1$ 个容器取 nxt 层，要求 $nxt \leq last$

如果状态太大，再优化成滚动数组或前缀最大值。

关键状态/数据结构

- $prefA[i][j]$ 、 $prefB[i][j]$ 或直接 $prefVal[i][j]$ 。
- $check(mid)$ 返回是否存在合法选择使贡献 ≥ 0 。
- 二分 60 次足够。

易错点

- 样例解释和输入格式可能矛盾；现场以正式输入和约束为准。
- 层的方向：题面说从下往上第 j 层，只能取下方若干层，就是取前缀。
- $sumB$ 必须为正才能谈比值；题面若没有保证，需要防御。
- 判断用 $best \geq -eps$ 。

组合拼装方案

- 第 03 卷翻 DP 五问和背包 DP。
- 第 12 卷复制“分数规划卡”的二分框架。
- 第 07 卷查浮点输出和自造样例： $N=1$ 、 $K=1$ 、所有 $b=1$ 。

考场写法建议

先写 DFS 版确认自己理解 k_i 约束；再写 $check(mid)$ ；最后套二分。不要在没验证判定函数时先调精度。

现场迭代路线

这题最关键的动机是：直接最大化 $sumA/sumB$ 不好 DP，但“能否达到某个比值 mid ”可以判定。考场上先写 DFS 枚举 k_i ，确认“只能取前缀”和“非增”方向没有写反；再把每层贡献改成 $a-mid*b$ 写 $check(mid)$ ；最后二分答案。若看到 $b=1$ 或总分母固定，先用普通最大和 DP 交特殊分。

代码卡片 A：部分分 DFS 枚举非增序列

合理化输入：N M，接着每个容器 M 层的 a b ，最后 K 。适用 N 小或 M 小的数据。

```
#include <bits/stdc++.h>
using namespace std;

int N, M, K;
vector<vector<double>> preA, preB;
double best = 0.0;

void dfs(int i, int lastTake, int used, double sumA, double sumB) {
    if (used > K) return;
    if (i == N + 1) {
        if (used == K && sumB > 0) best = max(best, sumA / sumB);
        return;
    }

    for (int take = 0; take <= lastTake && take <= M; take++) {
        dfs(i + 1, take, used + take, sumA + preA[i][take], sumB + preB[i][take]);
    }
}

int main() {
```

```

ios::sync_with_stdio(false);
cin.tie(nullptr);

if (!(cin >> N >> M)) return 0;
preA.assign(N + 1, vector<double>(M + 1, 0));
preB.assign(N + 1, vector<double>(M + 1, 0));

for (int i = 1; i <= N; i++) {
    for (int j = 1; j <= M; j++) {
        double a, b;
        cin >> a >> b;
        preA[i][j] = preA[i][j - 1] + a;
        preB[i][j] = preB[i][j - 1] + b;
    }
}
cin >> K;

dfs(1, M, 0, 0.0, 0.0);
cout << fixed << setprecision(6) << best << "\n";
return 0;
}

```

代码卡片 B: 部分分小 M 朴素二分 + DP 判定

适用: M 小, 先写清楚转移, 不做后缀最大优化。复杂度约 $O(\text{二次数} * N * K * M^2)$ 。

```

#include <bits/stdc++.h>
using namespace std;

const double NEG = -1e100;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M;
    if (!(cin >> N >> M)) return 0;

    vector<vector<double>> A(N + 1, vector<double>(M + 1, 0));
    vector<vector<double>> B(N + 1, vector<double>(M + 1, 0));
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= M; j++) cin >> A[i][j] >> B[i][j];
    }

    int K;
    cin >> K;

    vector<vector<double>> preA(N + 1, vector<double>(M + 1, 0));
    vector<vector<double>> preB(N + 1, vector<double>(M + 1, 0));
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= M; j++) {
            preA[i][j] = preA[i][j - 1] + A[i][j];
            preB[i][j] = preB[i][j - 1] + B[i][j];
        }
    }

    auto check = [&](double mid) {
        vector<vector<double>> dp(K + 1, vector<double>(M + 1, NEG));
        dp[0][M] = 0;

        for (int i = 1; i <= N; i++) {
            vector<vector<double>> ndp(K + 1, vector<double>(M + 1, NEG));
            for (int sum = 0; sum <= K; sum++) {
                for (int last = 0; last <= M; last++) {

```

```

        if (dp[sum][last] <= NEG / 2) continue;
        for (int take = 0; take <= last && sum + take <= K; take++) {
            double val = preA[i][take] - mid * preB[i][take];
            ndp[sum + take][take] = max(ndp[sum + take][take], dp[sum][last] + val);
        }
    }
    dp.swap(ndp);
}

double best = NEG;
for (int last = 0; last <= M; last++) best = max(best, dp[K][last]);
return best >= -1e-9;
};

double low = 0, high = 1;
for (int i = 1; i <= N; i++) {
    for (int j = 1; j <= M; j++) {
        if (B[i][j] > 0) high = max(high, A[i][j] / B[i][j]);
    }
}

for (int it = 0; it < 60; it++) {
    double mid = (low + high) / 2.0;
    if (check(mid)) low = mid;
    else high = mid;
}

cout << fixed << setprecision(6) << low << "\n";
return 0;
}

```

代码卡片 C: 部分分 b=1 特殊 DP

适用: 所有 b 都等于 1 或总 sumB 等于固定 K 的数据, 此时最大化比值等价于最大化 sumA。

```

#include <bits/stdc++.h>
using namespace std;

const double NEG = -1e100;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M;
    if (!(cin >> N >> M)) return 0;

    vector<vector<double>> preA(N + 1, vector<double>(M + 1, 0));
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= M; j++) {
            double a, b;
            cin >> a >> b;
            preA[i][j] = preA[i][j - 1] + a;
        }
    }

    int K;
    cin >> K;

    vector<vector<double>> dp(K + 1, vector<double>(M + 1, NEG));
    dp[0][M] = 0.0;

    for (int i = 1; i <= N; i++) {

```

```

vector<vector<double>> ndp(K + 1, vector<double>(M + 1, NEG));
for (int sum = 0; sum <= K; sum++) {
    for (int last = 0; last <= M; last++) {
        if (dp[sum][last] <= NEG / 2) continue;
        for (int take = 0; take <= last && take <= M && sum + take <= K; take++) {
            ndp[sum + take][take] = max(ndp[sum + take][take], dp[sum][last] + preA[i][take]);
        }
    }
}
dp.swap(ndp);
}

double ans = NEG;
for (int last = 0; last <= M; last++) ans = max(ans, dp[K][last]);
cout << fixed << setprecision(6) << (ans / max(1, K)) << "\n";
return 0;
}

```

代码卡片 D: 正解二分答案 + DP 判定

适用: 完整版本。最大比值用 $a\text{-mid} * b$ 转成判定, DP 用后缀最大优化上一容器取层数约束。

```

#include <bits/stdc++.h>
using namespace std;

const double NEG = -1e100;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M;
    if (!(cin >> N >> M)) return 0;

    vector<vector<double>> A(N + 1, vector<double>(M + 1, 0));
    vector<vector<double>> B(N + 1, vector<double>(M + 1, 0));

    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= M; j++) {
            cin >> A[i][j] >> B[i][j];
        }
    }

    int K;
    cin >> K;

    vector<vector<double>> preA(N + 1, vector<double>(M + 1, 0));
    vector<vector<double>> preB(N + 1, vector<double>(M + 1, 0));
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= M; j++) {
            preA[i][j] = preA[i][j - 1] + A[i][j];
            preB[i][j] = preB[i][j - 1] + B[i][j];
        }
    }

    auto check = [&](double mid) {
        vector<vector<double>> dp(K + 1, vector<double>(M + 1, NEG));
        dp[0][M] = 0.0;

        for (int i = 1; i <= N; i++) {
            vector<vector<double>> suffix(K + 1, vector<double>(M + 2, NEG));
            for (int sum = 0; sum <= K; sum++) {
                suffix[sum][M] = dp[sum][M];
                for (int last = M - 1; last >= 0; last--) {

```

```

        suffix[sum][last] = max(suffix[sum][last + 1], dp[sum][last]);
    }
}

vector<vector<double>> ndp(K + 1, vector<double>(M + 1, NEG));
for (int sum = 0; sum <= K; sum++) {
    for (int take = 0; take <= M && sum + take <= K; take++) {
        double bestPrev = suffix[sum][take];
        if (bestPrev <= NEG / 2) continue;
        double val = preA[i][take] - mid * preB[i][take];
        ndp[sum + take][take] = max(ndp[sum + take][take], bestPrev + val);
    }
}
dp.swap(ndp);
}

double best = NEG;
for (int last = 0; last <= M; last++) best = max(best, dp[K][last]);
return best >= -1e-9;
};

double low = 0.0, high = 0.0;
for (int i = 1; i <= N; i++) {
    for (int j = 1; j <= M; j++) {
        if (B[i][j] > 0) high = max(high, A[i][j] / B[i][j]);
    }
}
high = max(high, 1.0);

for (int it = 0; it < 70; it++) {
    double mid = (low + high) / 2.0;
    if (check(mid)) low = mid;
    else high = mid;
}

cout << fixed << setprecision(6) << low << "\n";
return 0;
}

```

C9. 2025 秋 3: 试剂资源实验选择

题目信号

- 每个实验最多一次。
- 消耗多种资源。
- 每种资源有容量。
- 最大化总价值。
- 题面数据范围不完整。

第一反应

对每个实验选或不选，DFS 枚举。

为什么朴素做法不够

M 稍大时 2^M 爆炸。若资源容量小，应该使用多维 0/1 背包；若容量大或维度多，需要剪枝、支配状态删除、折半。

模块路由

- 第 02 卷：DFS、剪枝、折半枚举。
- 第 03 卷：0/1 背包、多维背包。
- 第 07 卷：随机对拍。

部分分方案

1. $M \leq 25$: DFS 选/不选, 加剩余价值上界剪枝。
2. $N=1/2$ 且容量小: 一维/二维 0/1 背包。
3. 资源维度较多但可行状态数可控: 稀疏状态 + 支配删除。

正解推导

如果 N 小且容量 W_i 小, 把资源用量作为 DP 维度:

$dp[w1][w2]$ = 当前资源消耗下的最大价值

每个实验倒序转移, 保证 0/1。若 N 更多, 直接多维数组不可行, 可用 map 存稀疏状态, 并在每轮删除“资源消耗都不大于你、价值还更高”的支配状态。

关键状态/数据结构

- DFS: $idx, used[resource], value$ 。
- 背包: 多维容量状态。
- 稀疏状态: $vector<State>\{costs, value\}$ 。

易错点

- 第一行测试点编号可忽略, 但必须读取。
- 每个实验最多一次, 背包循环必须倒序或使用新旧数组。
- 价值可能需要 long long。
- 题面没有完整规模时, 先读数据估计容量和维度, 选择策略。

组合拼装方案

- 小数据复制第 02 卷 DFS。
- 容量小复制第 03 卷背包。
- 调试用第 07 卷对拍: DFS 和 DP 在小随机数据上比较。

考场写法建议

这类题先看 N 和 W_i 。如果容量总乘积过大, 不要硬开数组。先写 DFS 拿分, 再考虑 map 状态。

现场迭代路线

第一反应是 0/1 选择: 每个实验选或不选。先写 DFS, 立刻能拿小 M 分, 也能作为背包对拍器。然后看资源维度和容量: $N=1$ 用一维背包, $N=2$ 用二维背包; 如果容量乘积开不下, 就把“可行消耗向量”当稀疏状态保存, 并删除被支配状态。真正的现场决策点是“容量数组能不能开”, 不是盲目套多维 DP。

代码卡片 A: 部分分 DFS 选/不选

合理化输入: 第一行 testId 可忽略; 第二行 N M ; 第三行 N 个容量; 之后 M 行, 每行 N 个消耗和一个价值。适用 $M \leq 25$ 。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int N, M;
vector<int> W;
vector<vector<int>> cost;
vector<ll> val, suffixPositive;
vector<int> used;
ll ans = 0;

void dfs(int idx, ll cur) {
    if (idx == M + 1) {
        ans = max(ans, cur);
        return;
    }
    if (cur + suffixPositive[idx] <= ans) return;
```

```

    dfs(idx + 1, cur);

    bool ok = true;
    for (int r = 1; r <= N; r++) {
        if (used[r] + cost[idx][r] > W[r]) ok = false;
    }
    if (ok) {
        for (int r = 1; r <= N; r++) used[r] += cost[idx][r];
        dfs(idx + 1, cur + val[idx]);
        for (int r = 1; r <= N; r++) used[r] -= cost[idx][r];
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int testId;
    if (!(cin >> testId)) return 0;
    cin >> N >> M;

    W.assign(N + 1, 0);
    for (int r = 1; r <= N; r++) cin >> W[r];

    cost.assign(M + 1, vector<int>(N + 1, 0));
    val.assign(M + 1, 0);
    for (int i = 1; i <= M; i++) {
        for (int r = 1; r <= N; r++) cin >> cost[i][r];
        cin >> val[i];
    }

    suffixPositive.assign(M + 2, 0);
    for (int i = M; i >= 1; i--) suffixPositive[i] = suffixPositive[i + 1] + max(0LL, val[i]);

    used.assign(N + 1, 0);
    dfs(1, 0);
    cout << ans << "\n";
    return 0;
}

```

代码卡片 B: 部分分 N=1 一维背包

适用：只有一种资源且容量不太大。倒序循环保证每个实验最多选一次。

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int testId, N, M;
    if (!(cin >> testId)) return 0;
    cin >> N >> M;

    int W;
    cin >> W;

    vector<ll> dp(W + 1, 0);
    for (int i = 1; i <= M; i++) {
        int c;
        ll v;

```

```

        cin >> c >> v;
        for (int w = W; w >= c; w--) {
            dp[w] = max(dp[w], dp[w - c] + v);
        }
    }

    cout << *max_element(dp.begin(), dp.end()) << "\n";
    return 0;
}

```

代码卡片 C: 部分分 N=2 二维背包

适用: 两种资源且 $W1*W2$ 可开数组。

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int testId, N, M;
    if (!(cin >> testId)) return 0;
    cin >> N >> M;

    int W1, W2;
    cin >> W1 >> W2;

    vector<vector<ll>> dp(W1 + 1, vector<ll>(W2 + 1, 0));
    for (int i = 1; i <= M; i++) {
        int c1, c2;
        ll v;
        cin >> c1 >> c2 >> v;
        for (int a = W1; a >= c1; a--) {
            for (int b = W2; b >= c2; b--) {
                dp[a][b] = max(dp[a][b], dp[a - c1][b - c2] + v);
            }
        }
    }

    ll ans = 0;
    for (int a = 0; a <= W1; a++) {
        for (int b = 0; b <= W2; b++) ans = max(ans, dp[a][b]);
    }
    cout << ans << "\n";
    return 0;
}

```

代码卡片 D: 主力稀疏状态 + 支配删除

适用: 资源维度较多但可行状态数不爆的数据。状态数大时仍可能慢, 所以它是高分主力, 不保证所有极限数据满分。

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct State {
    vector<int> cost; // 1-index
    ll value = 0;
};

bool dominate(const State& a, const State& b, int N) {

```

```

    if (a.value < b.value) return false;
    for (int r = 1; r <= N; r++) {
        if (a.cost[r] > b.cost[r]) return false;
    }
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int testId, N, M;
    if (!(cin >> testId)) return 0;
    cin >> N >> M;

    vector<int> W(N + 1);
    for (int r = 1; r <= N; r++) cin >> W[r];

    vector<vector<int>> c(M + 1, vector<int>(N + 1, 0));
    vector<ll> v(M + 1, 0);
    for (int i = 1; i <= M; i++) {
        for (int r = 1; r <= N; r++) cin >> c[i][r];
        cin >> v[i];
    }

    vector<State> states;
    states.push_back({vector<int>(N + 1, 0), 0});

    for (int item = 1; item <= M; item++) {
        vector<State> all = states;

        for (const auto& st : states) {
            State ns = st;
            bool ok = true;
            for (int r = 1; r <= N; r++) {
                ns.cost[r] += c[item][r];
                if (ns.cost[r] > W[r]) ok = false;
            }
            if (ok) {
                ns.value += v[item];
                all.push_back(ns);
            }
        }

        vector<State> kept;
        vector<int> dead(all.size(), 0);
        for (int i = 0; i < (int)all.size(); i++) {
            if (dead[i]) continue;
            for (int j = 0; j < (int)all.size(); j++) {
                if (i == j || dead[j]) continue;
                if (dominate(all[i], all[j], N)) dead[j] = 1;
            }
        }
        for (int i = 0; i < (int)all.size(); i++) {
            if (!dead[i]) kept.push_back(all[i]);
        }
        states.swap(kept);
    }

    ll ans = 0;
    for (const auto& st : states) ans = max(ans, st.value);
    cout << ans << "\n";
    return 0;
}

```

}

C10. 2026 冬 1: 图片存储空间计算

题目信号

- $W \times H \times X$ 位。
- bit 转 byte, 再按 1024 转 KB/MB/GB。
- 题目指定四舍五入保留整数。
- $W, H \leq 1e6, X \leq 32$ 。

第一反应

直接算位数和字节数, 然后选择单位。

为什么朴素做法有风险

$W \times H \times X$ 最大约 $3.2e13$ 位, int 会爆。若以后数据更大, long long 也可能紧张, 使用 `__int128` 作为中间值更稳。

模块路由

- 第 01 卷: 整数 IO、格式输出。
- 第 11 卷: bit/byte/KB/MB/GB。
- 第 12 卷: 溢出卡。

部分分方案

用 long long 可过当前范围。小数据甚至 int 也可能过, 但不建议。

正解推导

先计算总 bit。字节数如果题面说不考虑不能整除, 要确认是否向上取整; 通常存储空间按字节需要:

$\text{bytes} = (\text{bits} + 7) / 8$

再按阈值选单位。KB/MB/GB 用 1024 进制。四舍五入整数可用 `floor(x + 0.5)` 或 `llround`。

关键状态/数据结构

`__int128 bits = (__int128)W * H * X;`

易错点

- bit 到 byte 是否向上取整, 按题面为准。
- 小于 1 KB 时输出 B, 不要输出 0 KB。
- 单位和数字之间一个空格。
- 输出整数, 不要多小数。

组合拼装方案

第 11 卷单位换算 + 第 01 卷输出。

考场写法建议

这题应快速拿下。自测 1 bit、8 bit、1023 B、1024 B、接近 1 MB 边界。

现场迭代路线

这题按单位链条一步步做, 不要跳步: 像素数 -> bit -> byte -> KB/MB/GB。第一版用 long long 已能覆盖常见范围; 如果题面给更大宽高, 马上换 `__int128`。最后再处理舍入规则, 因为“向上取整到字节”和“显示单位四舍五入”是两个不同位置的取整。

代码卡片 A: 部分分 long long 简洁版

适用: 当前范围 $W, H \leq 1e6, X \leq 32$ 。若未来范围更大, 换下面的 `__int128` 版。

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long W, H, X;
    if (!(cin >> W >> H >> X)) return 0;

    long long bits = W * H * X;
    long long bytes = (bits + 7) / 8;

    const long long KB = 1024LL;
    const long long MB = 1024LL * KB;
    const long long GB = 1024LL * MB;

    if (bytes < KB) cout << bytes << " B\n";
    else if (bytes < MB) cout << (bytes + KB / 2) / KB << " KB\n";
    else if (bytes < GB) cout << (bytes + MB / 2) / MB << " MB\n";
    else cout << (bytes + GB / 2) / GB << " GB\n";
    return 0;
}

```

代码卡片 B: 正解 ____int128 防溢出版

适用: 更大数据。用整数四舍五入, 不经过 double。

```

#include <bits/stdc++.h>
using namespace std;

string toString(__int128 x) {
    if (x == 0) return "0";
    string s;
    while (x > 0) {
        s.push_back(char('0' + x % 10));
        x /= 10;
    }
    reverse(s.begin(), s.end());
    return s;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long W, H, X;
    if (!(cin >> W >> H >> X)) return 0;

    __int128 bits = (__int128)W * H * X;
    __int128 bytes = (bits + 7) / 8;

    const __int128 KB = 1024;
    const __int128 MB = 1024 * KB;
    const __int128 GB = 1024 * MB;

    if (bytes < KB) {
        cout << toString(bytes) << " B\n";
    } else if (bytes < MB) {
        cout << toString((bytes + KB / 2) / KB) << " KB\n";
    } else if (bytes < GB) {
        cout << toString((bytes + MB / 2) / MB) << " MB\n";
    } else {
        cout << toString((bytes + GB / 2) / GB) << " GB\n";
    }
}

```

```

    }
    return 0;
}

```

C11. 2026 冬 2: 环形公路种树与连续段查询

题目信号

- 环形长度 $L \leq 1e9$ 。
- 操作数 $Q \leq 3e5$ 。
- 点赋值、区间清空、区间赋值、查询最长连续同类型段。
- 所有区间输入为闭区间。
- $l > r$ 表示跨过起点。

第一反应

用数组按每米维护树的类型。

为什么朴素做法不够

L 到 $1e9$ ，不能逐点数组。 Q 大，不能每次扫描区间。需要坐标压缩或动态线段树。

模块路由

- 第 04 卷：坐标压缩、线段树、懒标记区间赋值。
- 第 07 卷：闭区间/半开区间检查。
- 第 12 卷：环形区间线段树卡。

部分分方案

1. $L, Q \leq 1000$: 数组模拟。
2. 满分：坐标压缩 + lazy segment tree。若现场只需要中等数据，也可以按第 04 卷区间 map 思路改写，但本卷主给最稳的数组版和线段树版。

正解推导

核心是把闭区间 $[l, r]$ 统一转为半开区间 $[l, r+1)$ 。如果 $l > r$ ，拆成 $[l, L)$ 和 $[0, r+1)$ 。离线收集所有操作出现的边界：

$0, L, x, x+1, l, r+1$

相邻坐标形成一个叶子段，叶子的长度是 $\text{coord}[i+1] - \text{coord}[i]$ 。线段树维护每段上的颜色连续信息。清空可以用颜色 0，树类型为正整数。

节点字段：

len	当前段长度
leftColor	左端颜色
rightColor	右端颜色
pref	从左端开始的同色连续长度
suff	从右端开始的同色连续长度
best	段内最长同色连续长度
lazy	-1 表示无懒标记，否则整段赋成该颜色

合并两个节点 A+B：

```

res.len = A.len + B.len
res.leftColor = A.leftColor
res.rightColor = B.rightColor
res.pref = A.pref
if A.pref == A.len and A.rightColor == B.leftColor:
    res.pref = A.len + B.pref
res.suff = B.suff
if B.suff == B.len and A.rightColor == B.leftColor:
    res.suff = B.len + A.suff
res.best = max(A.best, B.best)
if A.rightColor == B.leftColor:
    res.best = max(res.best, A.suff + B.pref)

```

查询跨环区间时，不能简单 $\max(\text{left.best}, \text{right.best})$ ，必须按路径顺序合并： $\text{query}([l, L)) + \text{query}([0, r+1))$ 。

关键状态/数据结构

- `vector<long long> coord`: 压缩坐标。
- 线段树节点如上。
- 内部所有操作使用半开区间。

易错点

- 输入闭区间，要转 $r+1$ 。
- $r=L-1$ 时 $r+1=L$ ，必须提前放入坐标。
- 点赋值 x 等价于 $[x, x+1)$ 。
- 清空颜色 0 是否参与最长连续同类型？题面问连续同类型树木，通常空地不算树。若颜色 0 不应计入答案，则节点需要让空色 best 为 0；但合并时仍要维护空色边界。现场按题面判断。
- 样例 Q 可能矛盾，正式评测按输入 Q 行读取。

组合拼装方案

- 第 04 卷复制坐标压缩。
- 第 04 卷复制 lazy segment tree 的区间赋值骨架。
- 本卷复制节点字段和 merge 逻辑。
- 第 07 卷检查闭区间、跨环拆分、空色是否计数。

考场写法建议

先写数组模拟版并提交小数据；再实现节点 merge；最后接坐标压缩和懒标记。merge 函数单独用手造数据测试：两段同色、异色、左全同色、右全同色、跨环两段同色。

现场迭代路线

这题一定先用数组模拟理解操作语义：点赋值、区间清空、区间赋值、查询最长段分别怎么影响颜色。确认语义后，发现 L 太大，才引出坐标压缩；发现查询要跨左右边界合并，才引出线段树节点里的 `pref/suff/best`。现场升级顺序是“数组模拟交小分 -> 写 Info merge 并手测 -> 接懒标记 -> 最后处理环形拆分”，不要一上来把环形、压缩、懒标记、merge 全混在一起写。

代码卡片 A：部分分数组模拟

适用：L, Q ≤ 1000。颜色 0 表示空地，查询只统计非 0 树种。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int L, Q;
    if (!(cin >> L >> Q)) return 0;

    vector<int> color(L, 0);

    auto visitRange = [&](int l, int r, auto fn) {
        if (l <= r) {
            for (int x = l; x <= r; x++) fn(x);
        } else {
            for (int x = l; x < L; x++) fn(x);
            for (int x = 0; x <= r; x++) fn(x);
        }
    };

    for (int qi = 1; qi <= Q; qi++) {
        int type;
        cin >> type;
        if (type == 1) {
            int x, c;
            cin >> x >> c;
            color[x] = c;
        } else if (type == 2) {
```

```

        int l, r;
        cin >> l >> r;
        visitRange(l, r, [&](int x) { color[x] = 0; });
    } else if (type == 3) {
        int l, r, c;
        cin >> l >> r >> c;
        visitRange(l, r, [&](int x) { color[x] = c; });
    } else if (type == 4) {
        int l, r;
        cin >> l >> r;
        int best = 0, curLen = 0, curColor = -1;
        visitRange(l, r, [&](int x) {
            if (color[x] != 0 && color[x] == curColor) {
                curLen++;
            } else {
                curColor = color[x];
                curLen = (color[x] == 0 ? 0 : 1);
            }
            best = max(best, curLen);
        });
        cout << best << "\n";
    }
}
return 0;
}

```

代码卡片 B：主力坐标压缩 + 懒标记线段树

适用：坐标大、操作多。外部按题目坐标处理；压缩后的叶子下标封装在线段树内部，不需要和其它模块拼下标。

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

static constexpr bool COUNT_EMPTY = false;

static bool countedColor(int color) {
    return COUNT_EMPTY || color != 0;
}

struct Info {
    ll len = 0;
    int lc = -1, rc = -1;
    ll pref = 0, suff = 0, best = 0;
};

Info mergeInfo(const Info& a, const Info& b) {
    if (a.len == 0) return b;
    if (b.len == 0) return a;

    Info res;
    res.len = a.len + b.len;
    res.lc = a.lc;
    res.rc = b.rc;
    res.pref = a.pref;
    if (a.pref == a.len && a.rc == b.lc) res.pref = a.len + b.pref;
    res.suff = b.suff;
    if (b.suff == b.len && a.rc == b.lc) res.suff = b.len + a.suff;
    res.best = max(a.best, b.best);
    if (a.rc == b.lc && countedColor(a.rc)) {
        res.best = max(res.best, a.suff + b.pref);
    }
    return res;
}

```

```
}
```

```
struct SegTree {
    int n = 0;
    const vector<ll>* xs = nullptr;
    vector<ll> pref, suff, best;
    vector<int> lc, rc, lazy;

    void init(const vector<ll>& coord) {
        xs = &coord;
        n = (int)coord.size() - 1;
        int sz = 4 * max(1, n) + 5;
        pref.assign(sz, 0);
        suff.assign(sz, 0);
        best.assign(sz, 0);
        lc.assign(sz, 0);
        rc.assign(sz, 0);
        lazy.assign(sz, -1);
        if (n > 0) assignNode(1, 0, coord.back() - coord.front());
    }

    void assignNode(int p, int color, ll len) {
        lc[p] = rc[p] = color;
        pref[p] = suff[p] = len;
        best[p] = countedColor(color) ? len : 0;
        lazy[p] = color;
    }

    Info getInfo(int p, int l, int r) const {
        Info res;
        res.len = (*xs)[r + 1] - (*xs)[l];
        res.lc = lc[p];
        res.rc = rc[p];
        res.pref = pref[p];
        res.suff = suff[p];
        res.best = best[p];
        return res;
    }

    void setInfo(int p, const Info& v) {
        lc[p] = v.lc;
        rc[p] = v.rc;
        pref[p] = v.pref;
        suff[p] = v.suff;
        best[p] = v.best;
    }

    void push(int p, int l, int r) {
        if (lazy[p] == -1 || l == r) return;
        int mid = (l + r) / 2;
        int color = lazy[p];
        assignNode(p * 2, color, (*xs)[mid + 1] - (*xs)[l]);
        assignNode(p * 2 + 1, color, (*xs)[r + 1] - (*xs)[mid + 1]);
        lazy[p] = -1;
    }

    void pull(int p, int l, int r) {
        int mid = (l + r) / 2;
        Info left = getInfo(p * 2, l, mid);
        Info right = getInfo(p * 2 + 1, mid + 1, r);
        setInfo(p, mergeInfo(left, right));
        lazy[p] = -1;
    }
}
```

```

void update(int ql, int qr, int color) {
    if (ql > qr || n == 0) return;
    update(1, 0, n - 1, ql, qr, color);
}

void update(int p, int l, int r, int ql, int qr, int color) {
    if (ql <= l && r <= qr) {
        assignNode(p, color, (*xs)[r + 1] - (*xs)[l]);
        return;
    }
    push(p, l, r);
    int mid = (l + r) / 2;
    if (ql <= mid) update(p * 2, l, mid, ql, qr, color);
    if (qr > mid) update(p * 2 + 1, mid + 1, r, ql, qr, color);
    pull(p, l, r);
}

Info query(int ql, int qr) {
    if (ql > qr || n == 0) return Info();
    return query(1, 0, n - 1, ql, qr);
}

Info query(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return getInfo(p, l, r);
    push(p, l, r);
    int mid = (l + r) / 2;
    if (qr <= mid) return query(p * 2, l, mid, ql, qr);
    if (ql > mid) return query(p * 2 + 1, mid + 1, r, ql, qr);
    Info left = query(p * 2, l, mid, ql, qr);
    Info right = query(p * 2 + 1, mid + 1, r, ql, qr);
    return mergeInfo(left, right);
}

};

struct Operation {
    int type = 0;
    ll l = 0, r = 0;
    int color = 0;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    ll L;
    int Q;
    cin >> L >> Q;

    vector<Operation> ops;
    vector<ll> coord;
    ops.reserve(Q);
    coord.reserve(2 * Q + 2);
    coord.push_back(0);
    coord.push_back(L);

    for (int i = 0; i < Q; i++) {
        int type;
        cin >> type;
        if (type == 1) {
            ll x;
            int c;
            cin >> x >> c;

```

```

ops.push_back({type, x, x, c});
coord.push_back(x);
coord.push_back(x + 1);
} else if (type == 2) {
    ll l, r;
    cin >> l >> r;
    ops.push_back({type, l, r, 0});
    coord.push_back(l);
    coord.push_back(r + 1);
} else if (type == 3) {
    ll l, r;
    int c;
    cin >> l >> r >> c;
    ops.push_back({type, l, r, c});
    coord.push_back(l);
    coord.push_back(r + 1);
} else if (type == 4) {
    ll l, r;
    cin >> l >> r;
    ops.push_back({type, l, r, 0});
    coord.push_back(l);
    coord.push_back(r + 1);
}
}

sort(coord.begin(), coord.end());
coord.erase(unique(coord.begin(), coord.end()), coord.end());

auto idOf = [&](ll x) {
    return (int)(lower_bound(coord.begin(), coord.end(), x) - coord.begin());
};

SegTree seg;
seg.init(coord);

auto updateHalfOpen = [&](ll l, ll r, int color) {
    if (l >= r) return;
    int left = idOf(l);
    int right = idOf(r) - 1;
    seg.update(left, right, color);
};

auto queryHalfOpen = [&](ll l, ll r) {
    if (l >= r) return Info();
    int left = idOf(l);
    int right = idOf(r) - 1;
    return seg.query(left, right);
};

auto updateRing = [&](ll l, ll r, int color) {
    if (l <= r) {
        updateHalfOpen(l, r + 1, color);
    } else {
        updateHalfOpen(l, L, color);
        updateHalfOpen(0, r + 1, color);
    }
};

auto queryRing = [&](ll l, ll r) {
    if (l <= r) return queryHalfOpen(l, r + 1);
    Info left = queryHalfOpen(l, L);
    Info right = queryHalfOpen(0, r + 1);
    return mergeInfo(left, right);
};

```

```

};

for (const auto& op : ops) {
    if (op.type == 1) updateHalfOpen(op.l, op.l + 1, op.color);
    else if (op.type == 2) updateRing(op.l, op.r, 0);
    else if (op.type == 3) updateRing(op.l, op.r, op.color);
    else if (op.type == 4) cout << queryRing(op.l, op.r).best << "\n";
}

return 0;
}

```

C12. 2026 冬 3: 虚拟货币兑换的最小手续费

题目信号

- 兑换率矩阵。
- 套利环：乘积大于 1。
- 每条边乘以 $(1-p)$ 。
- 求最小手续费 p ，输出六位。
- $N \leq 100$ 。

第一反应

枚举所有环，算乘积是否大于 1。

为什么朴素做法不够

环数量指数级。乘积问题要取 $\log$ 转成加法，再用图上正环检测。

模块路由

- 第 05 卷：Floyd、Bellman-Ford、正环/负环思想。
- 第 06 卷： $\log$ 、浮点。
- 第 12 卷：套利与 $\log$ 图卡、浮点比较卡。

部分分方案

1. $N \leq 3$: 枚举长度 2、3 的环。
2. 固定 $p=0$ 检测套利：Floyd 乘积最大路径。
3. 满分：二分 p + 正环判定，或直接求最大环几何平均。

正解推导

原边权为兑换率 r_{ij} 。手续费后，每条边乘以 $(1-p)$ 。某个环可套利当且仅当：

$$\text{product}(r_e * (1-p)) > 1$$

取自然对数：

$$\text{sum}(\log(r_e) + \log(1-p)) > 0$$

给定 p ，所有边权为 $w_{ij} = \log(r_{ij}) + \log(1-p)$ ，判断图中是否存在正权环。若存在，手续费还不够；若不存在，手续费可行。随着 p 增大，所有边权变小，所以可二分最小 p 。

$N \leq 100$ ，判定可以用 Floyd 风格最大路径：初始化 $\text{dist}[i][j] = w[i][j]$ ，松弛后若存在 $\text{dist}[i][i] > \text{eps}$ ，说明有正环。也可以把边权取负后用负环检测。

满分可抄伪代码：

```

check(p):
    for i,j:
        d[i][j] = log(r[i][j]) + log(1-p)
    Floyd 最大路松弛:
        d[i][j] = max(d[i][j], d[i][k] + d[k][j])
    if any d[i][i] > eps:
        return false // 仍有套利，手续费不够
    return true

```

```
二分 p in [0, 1):
    if check(mid): right = mid
    else: left = mid
```

关键状态/数据结构

- double rate[105][105]。
- check(p): 返回是否无套利。
- 二分区间 [0,1), 迭代 80 次。

易错点

- $\log(1-p)$ 要求 $p < 1$ 。
- eps 不要太大, 正环判断可用 $1e-12$ 。
- 输出六位。
- 如果没有套利, 答案可能是 0。
- $r_{ii}=1$, 自环加手续费后小于 1, 不构成问题。

组合拼装方案

- 第 05 卷翻 Floyd/正环检测。
- 第 06 卷翻 log 和浮点输出。
- 本卷复制二分手续费判定逻辑。

考场写法建议

先写 $p=0$ 的套利检测, 确认样例能发现套利; 再套二分。不要直接乘积连乘, 容易溢出或下溢。

现场迭代路线

先不管手续费, 判断原始汇率是否有套利: 小 N 枚举 2/3 环, 大一点用 Floyd 最大乘积。确认“环乘积大于 1”之后, 再把乘法取 log 变成加法, 正乘积环就变成正权环。最后观察手续费 p 越大, 所有边权越小, 所以可以二分最小 p 。写代码时强制把 check(p) 命名为 noArbitrage, 避免二分方向写反。

代码卡片 A: 部分分 $N \leq 3$ 枚举小环

适用: 只需要判断没有手续费时是否存在 2 环或 3 环套利。完整矩阵输入。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<vector<double>> r(n + 1, vector<double>(n + 1));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) cin >> r[i][j];
    }

    bool ok = false;
    const double EPS = 1e-12;

    for (int a = 1; a <= n; a++) {
        for (int b = 1; b <= n; b++) {
            if (a == b) continue;
            if (r[a][b] * r[b][a] > 1.0 + EPS) ok = true;
        }
    }

    for (int a = 1; a <= n; a++) {
        for (int b = 1; b <= n; b++) {
```

```

        for (int c = 1; c <= n; c++) {
            if (a == b || b == c || a == c) continue;
            if (r[a][b] * r[b][c] * r[c][a] > 1.0 + EPS) ok = true;
        }
    }
}

cout << (ok ? "Arbitrage" : "No Arbitrage") << "\n";
return 0;
}

```

代码卡片 B: 部分分 p=0 最大乘积 Floyd

适用: 先判断原始汇率是否存在套利。若题目只问是否存在套利, 这就是主力。

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<vector<double>> d(n + 1, vector<double>(n + 1));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) cin >> d[i][j];
    }

    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                d[i][j] = max(d[i][j], d[i][k] * d[k][j]);
            }
        }
    }

    bool ok = false;
    for (int i = 1; i <= n; i++) {
        if (d[i][i] > 1.0 + 1e-12) ok = true;
    }
    cout << (ok ? "Arbitrage" : "No Arbitrage") << "\n";
    return 0;
}

```

代码卡片 C: 正解二分手续费 + log Floyd

适用: 完整矩阵输入; 手续费 p 是 $[0, 1]$ 小数; 兑换后每条边乘 $1-p$; 输出最小 p , 保留六位。

```

#include <bits/stdc++.h>
using namespace std;

const double EPS = 1e-12;

bool noArbitrage(double p, const vector<vector<double>>& lg, int n) {
    if (p >= 1.0) return true;

    double add = log1p(-p);
    vector<vector<double>> d(n + 1, vector<double>(n + 1));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            d[i][j] = lg[i][j] + add;
        }
    }
}

```

```

    }

    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                d[i][j] = max(d[i][j], d[i][k] + d[k][j]);
            }
        }
    }

    for (int i = 1; i <= n; i++) {
        if (d[i][i] > EPS) return false;
    }
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<vector<double>> lg(n + 1, vector<double>(n + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            double rate;
            cin >> rate;
            lg[i][j] = log(rate);
        }
    }

    if (noArbitrage(0.0, lg, n)) {
        cout << fixed << setprecision(6) << 0.0 << "\n";
        return 0;
    }

    double low = 0.0, high = 1.0;
    for (int it = 0; it < 80; it++) {
        double mid = (low + high) / 2.0;
        if (noArbitrage(mid, lg, n)) high = mid;
        else low = mid;
    }

    cout << fixed << setprecision(6) << high << "\n";
    return 0;
}

```

D. 专项算法卡片 (D1-D14)

D5. 分数规划卡

适用信号:

- 最大化或最小化一个比值: sumA/sumB 、平均值、单位收益。
- 选取方案有约束, 直接比较比值不好 DP。

转换:

$\text{sumA} / \text{sumB} \geq \text{mid}$
 $\Leftrightarrow \text{sumA} - \text{mid} * \text{sumB} \geq 0 \quad (\text{sumB} > 0)$

写法:

1. 二分 mid。
2. 把每个元素贡献改成 $a - mid * b$ 。
3. 用原问题约束做 DP/贪心/搜索，求最大改造贡献。
4. 若最大贡献 ≥ 0 ，说明 mid 可行，往右调。
5. 迭代 60 次或按题目精度停止。

常见坑：

- sumB 必须为正。
- check(mid) 里比较用 eps。
- 输出按题目精度，不要输出二分上下界中没更新的旧变量。

D6. 套利与 log 图卡

适用信号：

- 环路乘积是否大于 1。
- 汇率、概率、增长率连乘。
- 需要避免乘法溢出/下溢。

转换：

```
product(rate) > 1
<=> sum(log(rate)) > 0
```

手续费：

```
effective = rate * (1-p)
weight = log(rate) + log(1-p)
```

判定：

- 正环存在：仍可套利。
- 无正环：手续费足够。

可用模块：

- $N \leq 100$ ：Floyd $O(N^3)$ 。
- 边稀疏：Bellman-Ford/SPFA 风格正环检测。

D7. 环形区间线段树卡

统一规则：

- 输入闭区间 $[l, r]$ 。
- 内部全部转半开 $[l, r+1)$ 。
- 不跨环：直接处理 $[l, r+1)$ 。
- 跨环：拆成 $[l, L)$ 和 $[0, r+1)$ 。

查询跨环：

```
left = query(l, L)
right = query(0, r+1)
answerInfo = merge(left, right)
```

不要写成：

```
max(left.best, right.best)
```

因为最长段可能跨过 $L-1$ 和 0 的边界。

D8. 字符映射/单射卡

适用信号：

- 字符串同构。
- 模式匹配。
- 小写映射到大写。
- “不同字符不能映射到同一字符”。

标准状态声明：

```
int mp[26];
bool used[26];
```

函数内初始化片段：

```
fill(mp, mp + 26, -1);
fill(used, used + 26, false);
```

检查逻辑：

```
if mp[x] == -1:
    if used[y]: fail
    mp[x] = y
    used[y] = true
else:
    if mp[x] != y: fail
```

D9. 混合输入解析卡

适用信号：

- 一部分行是 $u \ v \ t$ 。
- 一部分行是 $u \rightarrow v \ t$ 。
- 行内分隔符不统一。
- 样例行数可能和 Q 矛盾。

原则：

- 已知后面要按行解析时，读完数字后立刻清换行；需要包含 `<limits>`：

```
cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

- 每条操作用 `getline` 读整行。
- 用 `find("->")` 判断特殊格式。
- 样例行数不可信时，正式代码仍按输入的 Q 行读取；若本地样例多一行，只能说明原样例有误，不要把代码改成读到 `EOF`。

D10. 浮点比较卡

常用写法：

```
fabs(a-b) <= eps    认为相等
a < b - eps         认为 a 明显小于 b
a > b + eps         认为 a 明显大于 b
```

二分：

- 输出 6 位：迭代 60 次通常够。
- `log` 前检查输入正数。
- 需要保留整数时，按题面用四舍五入、向上取整或向下取整，不要混用。

D11. 大整数/溢出卡

优先级：

1. 答案可能超过 `int`：用 `long long`。
2. 三个 $1e6$ 级数相乘或更大：中间用 `__int128`。
3. DP 负无穷：`const long long NEG = -(1LL<<60);`。
4. 最短路无穷：`const long long INF = (1LL<<60);`。

不要写：

```
int bits = W * H * X
```

要写：

```
__int128 bits = (__int128)W * H * X;
```

D12. 题面矛盾处理卡

常见矛盾：

- 样例截断。
- 样例 Q 与后续操作行数不一致。

- 题面提到参数，但输入格式没有。
- 数据范围前后冲突。
- 样例解释和输入格式不一致。

处理原则：

1. 不读取输入格式中不存在的参数。
2. 正式代码按输入格式和数据范围写，不按截断样例猜规则。
3. 若样例解释矛盾，优先相信题目定义和输出要求。
4. 代码尽量兼容边界，例如 $N=1$ 即使范围说 $N \geq 2$ 。
5. 不要在提交代码里写调试输出解释矛盾。

D13. 部分分优先卡

正解难点	先交版本
Dijkstra 写不稳	小图 Floyd / Bellman-Ford。
线段树写不完	小 L, Q 数组模拟。
复杂 DP 不会	DFS 枚举 + 剪枝 + 记忆化。
多维背包容量爆	DFS / map 稀疏状态 / 折半。
分数规划不会	枚举方案算比值，先过小数据。
套利最小手续费不会	先检测 $p=0$ 是否套利，再二分。

D14. 最后提交检查卡

- 没有调试输出。
- 没有 freopen 和文件 IO。
- 没有 #pragma。
- 没有联网、线程、系统调用。
- 所有答案和距离使用 long long 或更大。
- 数组大小足够，且多组数据会清空。
- 0/1-index 不混乱。
- 闭区间/半开区间统一。
- 浮点比较使用 eps。
- priority_queue.top() 前确认非空。
- 样例、自造边界都跑过。

E. 现场拼装总表

题卡	先交部分分	升级方向	必翻旧卷/专项卡
C1	公式 + 合法性	无	01, 11
C2	$O(nK)$ 扫描	堆 + 原编号	CPP-004, TRAIN-00
C3	DFS 小数据	集合 DP / 剪枝 / 费用流	02, 03, 05, 07
C4	while 模拟	无	01, 11
C5	乘积比较	eps	D10
C6	Floyd 小图	Dijkstra + 混合解析	CPP-011, GRAPH-03
C7	长度/一致性	单射映射	D8
C8	DFS 枚举	二分 + DP	D5, 03
C9	DFS	多维背包 / 稀疏状态	DP-24, 02, 07
C10	long long	__int128 和舍入	D11, 11
C11	数组模拟	坐标压缩 + lazy 线段树	D7, 04, 07
C12	小环枚举	log 图 + 二分正环	D6, 05, 06

F. 最后 30 分钟救分卡片

F1. 如果第三题完全没做

1. 读入写完整。

- 写小数据分支：
 - $N \leq 20$ DFS。
 - $N \leq 100$ Floyd。
 - $L, Q \leq 1000$ 数组模拟。
- 大数据不会时，优先提交小数据分支；只有题意允许空方案、合法构造或默认答案时，才输出格式合法的保守答案。
- 提交一次，不要让题目 0 提交。

F2. 如果 WA

优先查：

- 输出是否多空格/多行。
- 是否把闭区间当半开。
- 是否忘记 long long。
- 是否多组数据没清空。
- 是否样例本身有截断，自己按错误样例改坏了正式逻辑。

F3. 如果 TLE

快速降级：

- 图题：确认没有 $O(N^2)$ 找最短点，换堆。
- 区间题：确认不是每次扫全区间。
- DP：确认状态总数乘转移次数不爆。
- 搜索：加记忆化、排序剪枝、上界剪枝。

F4. 如果 RE

快速查：

- 数组下标是否到 n 但只开了 n 。
- vector 是否为空还访问。
- 递归深度是否太深。
- 坐标压缩后 lower_bound 结果是否越界。
- $r+1$ 是否可能超过 L 或数组范围。

F5. 如果最后只剩 5 分钟

- 不重写算法。
- 删除调试输出。
- 确认每题最高分版本已提交。
- 对未提交题，优先提交小数据版本；只有题意允许明确合法兜底时，才提交保守输出。
- 不在最后一刻改核心边界。

G. 题内代码索引

本卷不再把完整代码单独堆在最后，避免同一道题出现多份不同版本。考试时直接翻对应 C 题：每题内部已经按“部分分代码卡片 -> 主力/正解代码卡片”的顺序放好完整可运行程序。

题目	题内代码卡片
C1 三角形面积	先交版/正解版
C2 路由器优先级调度	A: $O(nK)$ 线性扫描; B: priority_queue 主力版
C3 自动驾驶车队调度	A: DFS; B: 集合 DP/记忆化; C: 贪心兜底
C4 线性递推超过阈值	先交版/正解版
C5 最优汇率转换	A: 直接比较; B: eps 正解
C6 虫洞穿梭	A: Floyd; B: Bellman-Ford; C: Dijkstra
C7 字符串嵌入判定	A: 只查长度; B: 一致性; C: 一致性 + 单射
C8 药液比例最大化	A: DFS 枚举; B: 小 M 朴素二分 + DP; C: $b=1$ 特殊 DP; D: 后缀优化二分 + DP
C9 试剂资源实验选择	A: DFS; B: $N=1$ 背包; C: $N=2$ 背包; D: 稀疏状态
C10 图片存储空间计算	A: long long; B: __int128
C11 环形公路种树	A: 数组模拟; B: 坐标压缩 + 懒标记线段树

题目	题内代码卡片
C12 虚拟货币最小手续费	A: 小环枚举; B: p=0 Floyd; C: 二分 + log Floyd